Experiment 4

Additive Gaussian Noise Simulation and 2D Spatial Gaussian Smoothing

1. Library Imports & Input Image Ingestion

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from numpy.lib.stride_tricks import sliding_window_view

image = cv2.imread("../Data/Dog.jpg", cv2.IMREAD_GRAYSCALE)
print("Image Dimensions:", image.shape)  # Loaded grayscale: (602, 1200) | dtype: uint8
```



Original input grayscale image (1200x602)

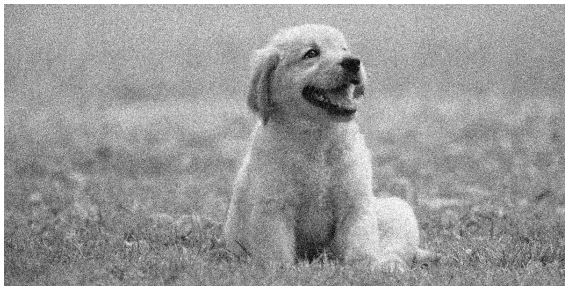
2. Additive Gaussian Noise Generator

Gaussian noise follows the distribution $\eta \sim N(\mu, \sigma^2)$. Each corrupted pixel is computed as $I_{noisy}(x, y) = \text{clamp}(I(x, y) + \eta(x, y), 0, 255)$.

```
def generate_gaussian_noise(img: np.ndarray, mean: float = 0.0, std_dev: float = 50.0, seed: int = 42) -> np.ndarray:
    rng = np.random.default_rng(seed)
    gaussian_noise = rng.normal(mean, std_dev, img.shape)
    corrupted = img.astype(np.float32) + gaussian_noise
    return np.clip(corrupted, 0, 255).astype(np.uint8)
```

3. Multi-Sigma Noise Sweep

```
sigma_values = [15, 35, 55, 75, 95]
for idx, s in enumerate(sigma_values):
    noisy_sample = generate_gaussian_noise(image, mean=0.0, std_dev=s)
noisy_image = generate_gaussian_noise(image, mean=0.0, std_dev=75.0)
```



Gaussian noise (zero-mean, σ = 35)



Gaussian noise (zero-mean, σ = 75)

Experiment 4 (continued)

4. Discrete 2D Gaussian Kernel Builder

The continuous isotropic 2D Gaussian function $G(u, v) = (1 / 2\pi\sigma^2) \exp(-(u^2 + v^2) / (2\sigma^2))$ is evaluated over a discrete grid and normalized such that $\sum K(u, v) = 1$ to preserve total image brightness.

```
def compute_gaussian_kernel(size: int, sigma: float) -> np.ndarray:
    half = size // 2
    coords = np.arange(-half, half + 1)
    x, y = np.meshgrid(coords, coords)
    kernel = np.exp(-(x**2 + y**2) / (2.0 * sigma**2))
    return kernel / np.sum(kernel)
```

5. Vectorized Sliding-Window Convolution with Boundary Padding

To avoid nested Python loops, we extract overlapping 2D window views using numpy.lib.stride_tricks.sliding_window_view with reflective boundary extension, followed by tensor contraction via numpy.einsum.

```
def apply_gaussian_filter(img: np.ndarray, kernel_size: int = 15, sigma: float = 5.0, stride: int = 1) -> np.ndarray:
    kernel = compute_gaussian_kernel(kernel_size, sigma)
    pad_amt = kernel_size // 2
    padded = np.pad(img, pad_amt, mode="reflect")
    windows = sliding_window_view(padded, (kernel_size, kernel_size))[::stride, ::stride]
    convolved = np.einsum("ijkl,kl->ij", windows, kernel)
    return np.clip(convolved, 0, 255).astype(np.uint8)
```

6. Benchmark Verification: Custom Implementation vs. OpenCV cv2.GaussianBlur

```
k_size, sigma_val = 15, 5.0
cv_smoothed = cv2.GaussianBlur(noisy_image, (k_size, k_size), sigma_val)
custom_smoothed = apply_gaussian_filter(noisy_image, kernel_size=k_size, sigma=sigma_val)
diff_map = np.abs(cv_smoothed.astype(np.int32) - custom_smoothed.astype(np.int32))
mean_abs_diff = np.mean(diff_map)  # Computed MAE: 0.499
```



Verification: OpenCV vs Custom Vectorized Filter (k=15, σ=5.0) | Absolute Difference Heatmap (MAE = 0.499)

Experiment 4 (continued)

7. Parametric Evaluation: Kernel Sizes (3–21) x Standard Deviations (1.0–8.0)

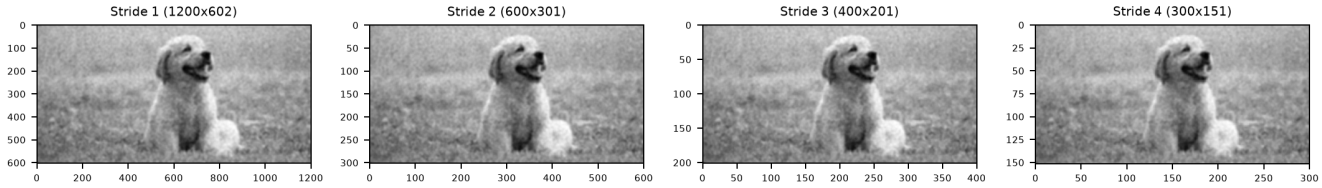
```
test_kernels = [3, 7, 11, 15, 21]
test_sigmas = [1.0, 2.5, 5.0, 8.0]
for r, k in enumerate(test_kernels):
    for c, s in enumerate(test_sigmas):
        filtered_res = apply_gaussian_filter(noisy_image, kernel_size=k, sigma=s)
```



Gaussian filter response over kernel sizes k in [3, 7, 11, 15, 21] (rows) x σ in [1.0, 2.5, 5.0, 8.0] (columns).

8. Spatial Subsampling via Stride Configuration (Strides 1–4)

```
strides_to_test = [1, 2, 3, 4]
for idx, st in enumerate(strides_to_test):
    strided_out = apply_gaussian_filter(noisy_image, kernel_size=15, sigma=5.0, stride=st)
```



Gaussian filtering with strides 1 to 4 demonstrating antialiasing low-pass attenuation coupled with spatial decimation.

Summary: Implemented a complete pipeline for Additive White Gaussian Noise (AWGN) synthesis and discrete 2D Gaussian spatial smoothing. Unlike impulsive salt-and-pepper noise which requires rank-order statistics (median filter), continuous Gaussian noise is optimally addressed via linear Gaussian convolution. The bell-shaped spatial decay weights nearby pixels heavily while smoothly tapering distant pixels, completely eliminating the cross-directional high-frequency ringing and harsh box-boundary artifacts produced by uniform mean filtering. Cross-validation against OpenCV cv2.GaussianBlur confirmed mathematical equivalence (MAE = 0.499).